

二进制与随机逻辑混合计算

Hongge Li, IEEE 会员, 以及 Yuhao Chen, IEEE 研究生会员

摘要

二进制逻辑广泛应用于数字信号处理和计算机系统, 因其可直接在 CMOS 电路中实现. 随机逻辑采用特殊的数据表示方式, 以逻辑电平为 ON 的概率表示数据. 随机逻辑计算是基于随机比特流而非二进制数的计算类型. 本文提出一种二进制逻辑与随机逻辑的混合计算系统, 称为混合逻辑计算. 文中讨论了混合逻辑电路的生成方法, 并分析其特性.

关键词: 现场可编程门阵列 (FPGA), 混合逻辑计算, 低延迟, 随机计算 (SC)

1 引言

随机逻辑的特性一直是学术界的研究热点. 随机逻辑本质上是一个基于概率与均值的数学问题, 利用简单逻辑电路即可实现具有特殊性质的随机逻辑功能, 从而替代基于二进制逻辑的复杂电路. 另一方面, 在二进制逻辑 (布尔代数) 中, 系统任意节点的变量仅能取两个值之一, 记为 0 或 1.

随着计算规模增大, 传统数字计算的功耗需求随之增加, 导致计算规模与求解速度之间需要权衡. 为此, 研究者提出了一种新的数据处理系统——随机计算 (SC), 该系统最大化利用各逻辑门进行计算, 并以分布式方式在空间和时间上执行运算.

二进制逻辑在执行数字计算时具有显著的速度优势. SC 是由 Gaines 于 1969 年提出的一种重要脉冲计算方法 [1]. 在 SC 范式中, 计算基于随机比特流 (SB) 而非二进制数 (BN) 进行. SB 通过比特流中“1”的出现概率表示数值, 该概率基于累积时间计算 [1],[2]. SC 所需的计算能力完全分布于低成本的逻辑门中. 二进制逻辑电路采用高效的数据编码方式, 仅需 $\log_2 N$ 个有序二进制电平. 相反, SC 是数据表示效率最低的方式, 需要 N^2 个无序二进制电平 [1]. 然而, 传统 SC 以牺牲效率为代价, 换取了大量硬件资源和功耗的节省.

在 SC 范式中, 数字计算基于 SB 而非 BN 进行, 这在处理复杂函数时具有显著的硬件开销优势. SC 已广泛应用于多个领域 [2],[3], 包括图像处理 [4]、通信编码 [5]、神经网络 [6] 和深度学习 [7]. Sim 等人提出了新的乘法方法, 用于缩短流长度并优化高延迟问题. Najafi 和 Lilja[8] 提出了确定性方法以实现精确计算. Parhi[9] 提出了单极性与双极性混合格式, 以获得更好的能效. Liu 和 Han[10] 采用 Sobol 序列优化精度不足的问题. Canals 等人 [11] 提出了一种新的编码方法, 以分数形式用两组 SB 表示一个值, 将脉冲表示范围扩展至整个实数轴.



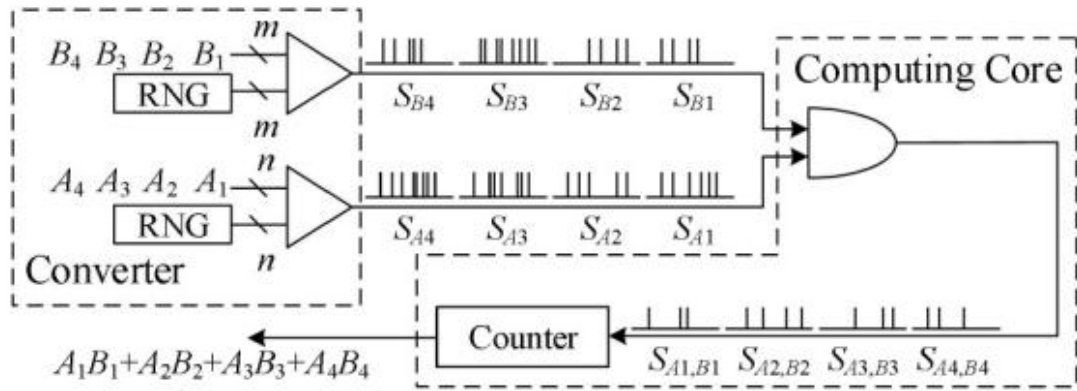


图 1: 基于 SB 的经典 SC

传统 SC 的高延迟本质上源于 1 比特随机流较低的信息携带能力。当随机逻辑的比特流长度为 N 时，其仅能表示 N 个不同的数值。显然，比特流的信息携带效率低于 BN。为提高 SC 编码效率，研究者尝试了多种方法，如使用多比特流缩短延迟 [12],[13]，或采用温度计编码器 [14]。

尽管 SC 在硬件开销和噪声容限方面具有优势，但高延迟和低精度仍是 SC 硬件电路设计面临的主要挑战。为解决这些根本性问题，本文提出了一种新的计算方法。主要贡献如下：

1. 提出了二进制逻辑和随机逻辑的混合逻辑计算，用于混合逻辑 SC(HSC)。
2. 讨论了混合、随机和二进制逻辑的性质和关系。
3. 设计了乘法器、加法器、除法器以及 BN 和 SB 转换器的混合逻辑电路。

2 随机计算

2.1 现有 SC 方法

基于随机比特流的经典 SC 乘法 [1] 如图1所示。随机比特流生成器由随机数生成器 (RNG) 和比较器组成。通过比特流生成器， A 和 B 分别被转换为随机比特流 S_A 和 S_B 。两个流为“1”的概率分别为 $P(S_A) = A/2^n$ 和 $P(S_B) = B/2^m$ 。经“AND”门后得到新的随机比特流 $S_{A,B}$ 。输出流为“1”的概率为 $P(S_{A,B}) = P(S_A)P(S_B) = AB/2^{n+m}$ ，实现了两个输入流 S_A 和 S_B 的乘法。计数器逐个累加四个随机流 $S_{A1,B1}$ 至 $S_{A4,B4}$ ，以获得 $A_1B_1 + A_2B_2 + A_3B_3 + A_4B_4$ 的完整结果。

SC 结果可能存在误差的原因如下。首先，两个输入流须相互独立；否则 $P(S_{A,B}) = P(S_A)P(S_B)$ 不成立。其次，随机流本质上是 N_F 重伯努利试验的结果，流中“1”的数量服从二项分布 [11]。SB 的长度随所需分辨率呈指数增长。例如，要获得具有 8 位分辨率的流概率，使用经典 SC 须观察至少 2^{16} 比特的流长度 [15]。

2.2 随机与二进制混合逻辑

SC 通常以流中“1”的概率表示数值. 设 $x(1), x(2), \dots, x(N)$ 为该流. 比特流中的每个 $x(i)$ 由离散随机变量 X 生成, 其概率分布为 $P(X=1)=p$ 和 $P(X=0)=1-p$, 其中 p 为概率值. 根据已有研究中期望的定义, X 的期望可由下式求得 [17]:

$$E[X] = 1 \cdot p + 0 \cdot (1-p) = p. \quad (1)$$

由 (1) 可知, SC 亦可理解为使用离散随机变量 X 作为信号源生成随机流, 且 $E[X]$ 表示该流的值. 由于在传统 SC 中 X 取值为 1 或 0, 故随机脉冲为 1 比特流.

按上述方式理解, 离散随机变量的取值范围可进一步扩展, 而非局限于 1 比特流. 因此, SC 可扩展为使用多比特流替代 1 比特流. 图2和图3展示了在 2 比特流上进行计算的示例. 由于这种新的随机流兼具传统随机逻辑与二进制逻辑的特性, 本文将新方法称为随机与二进制的混合逻辑计算, 并将多比特随机流称为混合流.

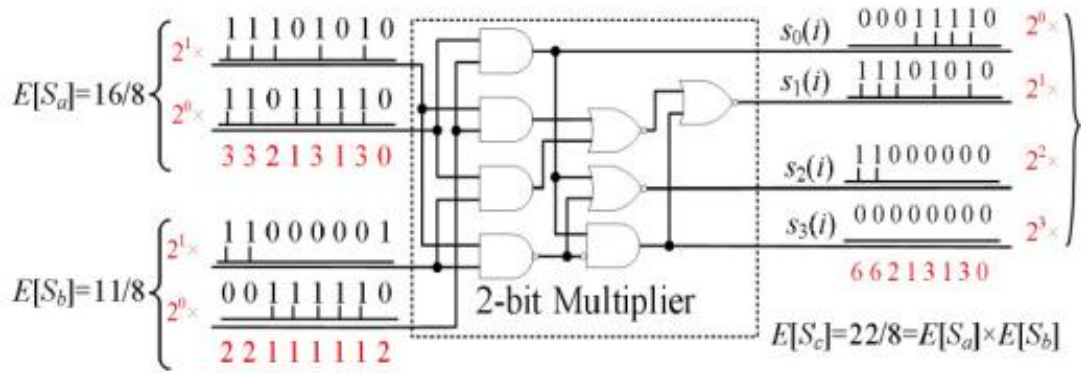


图 2: HSC 的乘法

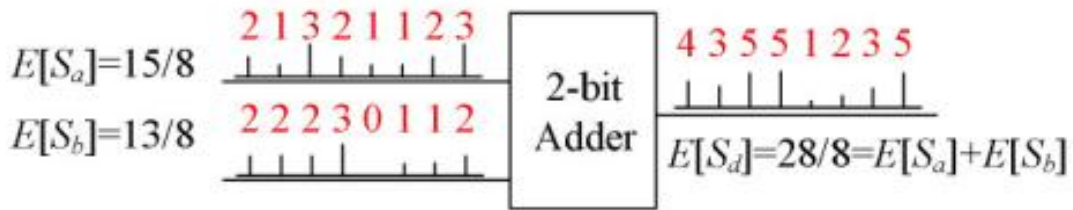


图 3: HSC 的加法

1) 乘法和加法: 将混合流 $s_a(i)$ 和 $s_b(i)$ 逐元素相乘, 可得新的混合流 $s_c(i)$

$$s_c(i) = s_a(i)s_b(i). \quad (2)$$

根据概率理论, 若 $s_a(i)$ 和 $s_b(i)$ 相互独立, 则 $s_c(i)$ 的期望为 $E[S_a]$ 与 $E[S_b]$ 之积, 如 (3) 所示.

因此, (2) 实现了两个混合流的乘法运算.

$$E[S_c] = E[S_a S_b] = E[S_a] E[S_b]. \quad (3)$$

将两个混合流 $s_a(i)$ 和 $s_b(i)$ 逐元素相加, 可得新的混合流 $s_d(i)$

$$s_d(i) = s_a(i) + s_b(i). \quad (4)$$

根据概率定理, $s_d(i)$ 的期望等于 $E[S_a] + E[S_b]$, 如 (5) 所示. 因此, (4) 实现了两个混合流的加法运算.

$$E[S_d] = E[S_a + S_b] = E[S_a] + E[S_b]. \quad (5)$$

下面以图2为例说明细节. 首先将 BN 16 和 11 缩放至 $[0, 2^2 - 1]$ 范围, 以满足 2 比特混合流的表示范围. 由此, 2 比特流以 $E[S_a] = 16/8$ 和 $E[S_b] = 11/8$ 编码, 分别表示 16 和 11. 采用这种混合逻辑, 可使用 2 比特乘法器实现 n 比特 ($n \geq 2$) 二进制乘法, 如图2所示; 使用 2 比特加法器实现 n 比特 ($n \geq 2$) 加法, 如图3所示.

混合逻辑流不仅由 2 比特组成. 当使用 m 比特混合流表示 n 比特二进制数据 ($m \leq n$) 时, 可以使用 m 比特乘法器进行 n 比特二进制乘法.

2) 除法: 图4(a) 展示了 HSC 除法电路结构. 其中, $s_a(i)$ 和 $s_b(i)$ 为输入混合流, $i = 0, 1, \dots, N-1$; 除法结果 $s_c(i)$ 亦为混合流. 计数器用于计算 $\sum_{i=1}^n |\overline{s_a(i)} - s_c(i) s_b(i)|$. 当计数器的期望等于 0 时, $E[S_c S_b] = E[S_a]$ 成立. 在此条件下, 若 S_c 和 S_b 相互独立, 则有 $E[S_c] E[S_b] = E[S_a]$. 因此, 当计数器趋近于 0 时, $s_c(i)$ 即趋近于除法结果. 将计数器和 $s_c(0)$ 的初始值均设为零, 并使用 RNG 控制 $s_c(i)$ 的值: 当计数器 > 0 时, $s_c(i) = \text{RNG}$; 当计数器 < 0 时, $s_c(i) = 0$. 除法电路的精度测试如图4(b) 所示, 当 $E[S_a] = 3$ 且 $n = m = l = 3$ 时, $E[S_c] = 3/E[S_b]$. 除法电路的数据流示例如图4(c) 所示.

3) BN-SB 转换器: 本文提出了两种类型的 BN-SB 电路, 即 A 型和 B 型, 如图5所示. BN-SB (A 型) 以一个 BN x (n 比特) 作为输入来编码多比特流. 例如, 图5(a) 展示了 m 比特随机流的编码过程, 其中 x_i 表示 x 的第 i 位. x 的低 $(n - m + 1)$ 位与 RNG 比较以获得比特流 $s_0(i)$; x 的高 $(m - 1)$ 位分别与 1 比特 RNG 比较, 以获得 $s_1(i), s_2(i), \dots, s_{m-1}(i)$. 此处, 1 比特 RNG 可从 $(n - m + 1)$ 比特 RNG 的任意位获取. 在图5(a) 中, 以 $m = 3$ 为例, 3 比特流输出与输入 x 的关系如下:

$$E[S_0] = p_0 = (2^{n-3} x_{n-3} + \dots + 2^1 x_1 + x_0) / 2^{n-2} \quad (6)$$

$$E[S_i] = p_i = x_{n-3+i} / 2, \quad i = 1, 2. \quad (7)$$

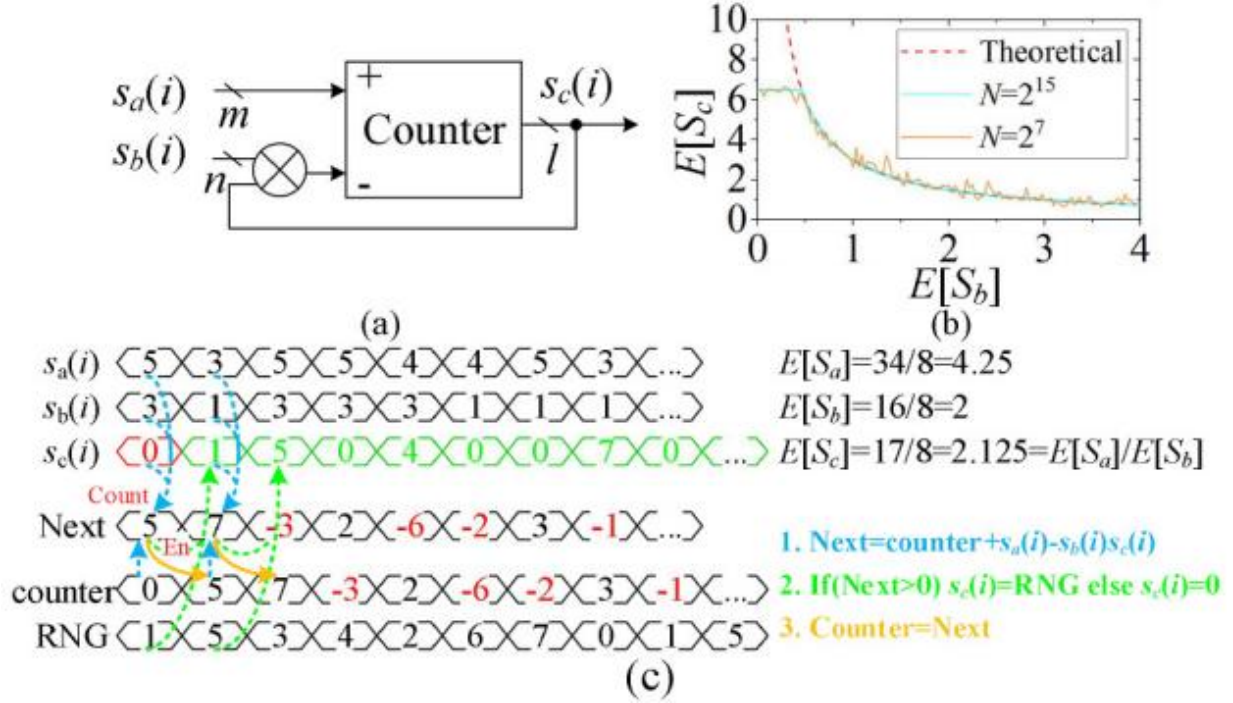


图 4: HSC 的除法. (a) 除法计算电路. (b) 除法电路的精度评估, $E[S_c] = 3/E[S_b]$. (c) 除法电路示例.

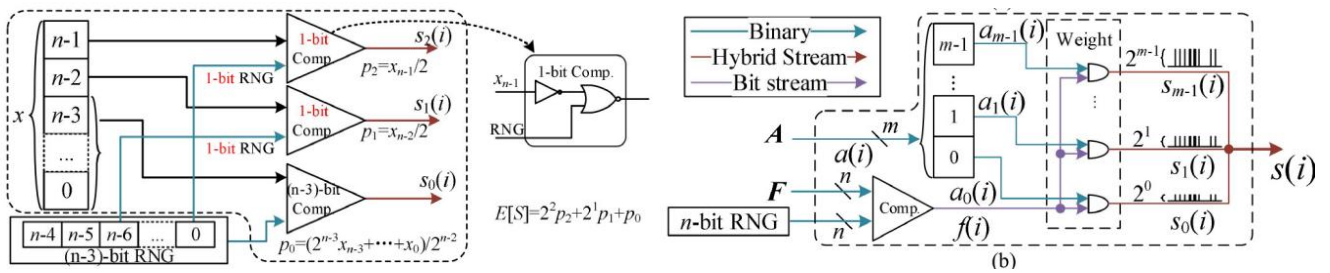


图 5: 混合逻辑计算的 BN-SB 转换器. (a)A 型使用一个二进制数据编码 3 比特流. (b)B 型使用两个二进制数据编码混合流.

因此, BN-SB (A 型) 输出的 3 比特流期望与 x 成正比, 实现了 BN x 的多比特流编码

$$\begin{aligned} E[S] &= 2^2 p_2 + 2^1 p_1 + 2^0 p_0 \\ &= (2^{n-1} x_{n-1} + 2^{n-2} x_{n-2} \dots + x_0) / 2^{n-2} = x / 2^{n-2}. \end{aligned} \quad (8)$$

B 型电路如图5(b) 所示, 是另一种编码多比特流的方法, 使用两个 BN 编码一个多比特流. 输出的多比特流表示两个输入 BN 的乘积. 待编码的 BN x ($n + m$ 比特) 为 y 与 p 的乘积, 即 $x = y \cdot p = y \cdot z / 2^n$. 其中, z 为 n 比特, y 为 m 比特, 且 $p \in [0, 1]$. 通过 z 与随机数的比较生成表示 p 的比特流 $f(i)$, 这与传统 SC 的 BN-SB 电路相同

$$E[F] = p = z / 2^n. \quad (9)$$

此后, 若 $a(i) = y$ 且 $i = 1, 2, \dots, N$, 则 $E[A] = y$.

最后, 将 $a(i)$ 和 $f(i)$ 相乘得到 $s(i)$, 如图5右侧所示. 其中, $a_0(i), a_1(i), \dots, a_{m-1}(i)$ 表示 $a(i)$ 的 m 比特. $s(i)$ 的 m 比特与 $a(i)$ 具有相同的表示方式

$$s(i) = \sum_{j=0}^{m-1} 2^j s_j(i) = \sum_{j=0}^{m-1} 2^j [a_j(i) \cdot f(i)]. \quad (10)$$

$s(i)$ 表示的值由其期望决定. 因 A 为常数, 由 (11) 可得期望. 因此, $s(i)$ 为表示 x 的多比特流, 该电路实现了将 BN x 转换为多比特流的功能

$$E[S] = \left(\sum_{j=0}^{m-1} 2^j y_j \right) E[F] = y \cdot E[F] = y \cdot p = x. \quad (11)$$

与传统比特流相比, 混合逻辑计算对流长度 N 的要求更低即可达到相同的分辨率. 将 n 比特 BN x 转换为 $(m+1)$ 比特流时, $(m+1)$ 比特流的每个比特流具有 $(n-m)$ 比特分辨率 ($N = 2^{n-m} - 1$), 足以表示 x_i ; 而经典 SC 需要 n 比特分辨率 ($N = 2^n - 1$).

当 $m = 0$ 时, 混合逻辑流退化为经典比特流. 这是一种极端情况, 具有最低的硬件开销但最差的延迟. 当 $m = n - 1$ 时, 混合逻辑流转换为 BN. 这是另一种极端情况, 具有最佳的延迟和最大的硬件开销. 通过这种混合逻辑, 可在延迟与硬件开销之间进行权衡, 而非仅限于两种极端情况之间的选择.

3 混合逻辑计算讨论

3.1 混合逻辑的噪声容限

在噪声环境中，假设随机流的每个比特具有相同的比特翻转率 $a \in [0, 0.5]$. 此外，比特流 $x(i)$ 由离散随机变量 X 生成， X 的期望为 $p \in [0, 1]$. X_{noise} 表示受噪声影响的随机变量. $x(i)$ 比特翻转的四种情况如下：

1. 发送 1 得到 1, $p_1 = p(1 - a)$
2. 发送 1 得到 0, $p_2 = pa$
3. 发送 0 得到 1, $p_3 = (1 - p)a$
4. 发送 0 得到 0, $p_4 = (1 - p)(1 - a)$.

由此可得 X_{noise} 的期望：

$$E[X_{\text{noise}}] = p_1 + 0 \cdot p_2 + p_3 + 0 \cdot p_4 = (1 - 2a)E[X] + a. \quad (12)$$

SC 的特性表明， X_{noise} 的期望与 X 之间存在确定的线性关系，该关系与比特翻转率 a 相关. 该特性使 SC 具有高噪声容限. 需要注意的是，当 $a = 0.5$ 时， $E[X_{\text{noise}}] = 0.5$ ，此时 X_{noise} 与 X 无关，信息完全丢失.

m 比特流 $s(i)$ 可视为比特流 $s_0(i), \dots, s_{m-1}(i)$ 的并行组合，且 (13) 亦适用于 $s_0(i), \dots, s_{m-1}(i)$. 假设随机流的每个比特具有相同的比特翻转率 a ，且 $E[S_i] = p_i, i = 0, 1, \dots, m - 1$. s 的期望为

$$E[S] = 2^{m-1}E[S_{m-1}] + 2^{m-2}E[S_{m-2}] + \dots + 2^0E[S_0]. \quad (13)$$

由 (12) 和 (13) 可得，受噪声影响的 m 比特流 $s(i)$ 的期望如 (14) 所示. 比特翻转误差同样导致混合流期望的线性变化，便于恢复原始数据. 因此，混合流具有与比特流相同的噪声容限. HSC 基于比特流、多比特流和二进制数据的噪声容限如图6所示.

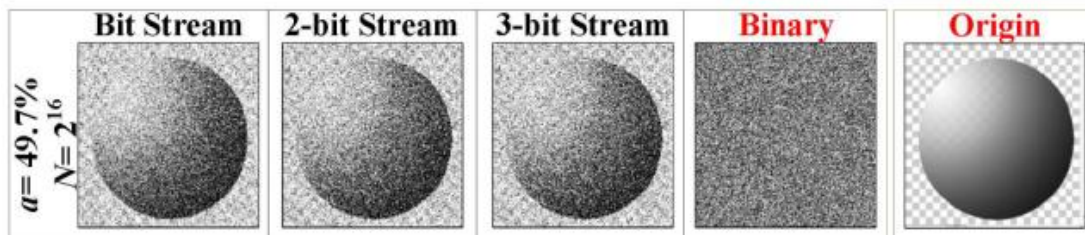


图 6: HSC 噪声容限实验

$$\begin{aligned}
E[S_{\text{noise}}] &= 2^{m-1}E[S_{m-1,\text{noise}}] + \dots + 2^0E[S_{0,\text{noise}}] \\
&= (1-2a)(2^{m-1}p_{m-1} + \dots + 2^0p_0) + a(2^{m-1} + \dots + 2^0) \\
&= (1-2a)E[S] + a(2^m - 1).
\end{aligned} \tag{14}$$

3.2 混合逻辑的精度实验

渐进精度 [16] 的存在使得通过选择不同流长度即可灵活调整精度, 这是 SC 的优点之一. 由于真 RNG 的硬件实现较为复杂, SC 中通常使用伪随机序列. 使用伪随机序列时, 通过统计方法获得渐进精度是更优策略, 以下测试均采用该策略.

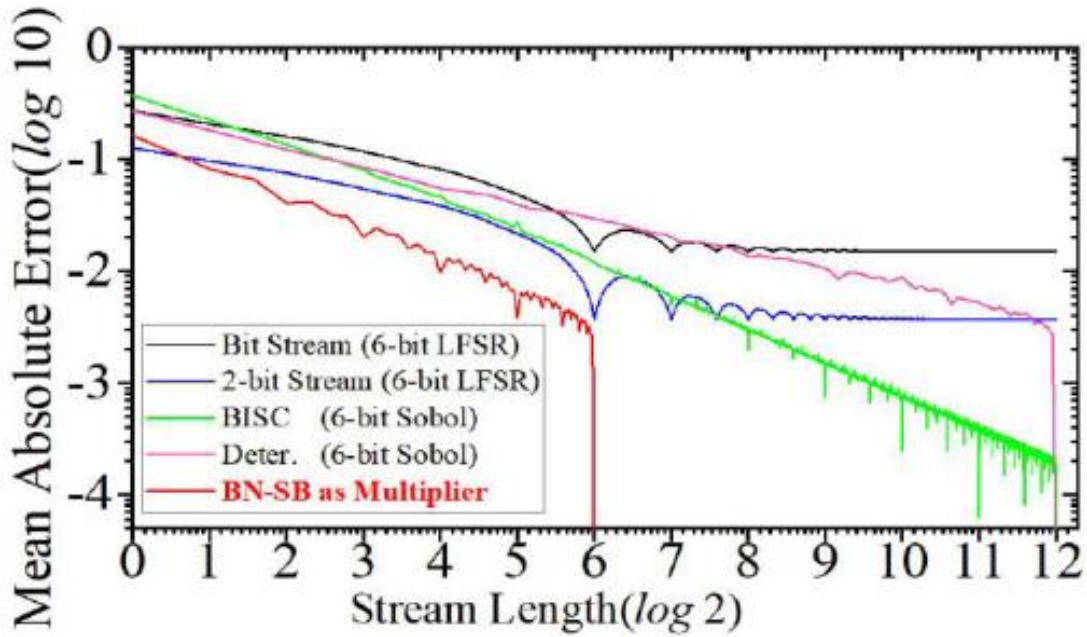


图 7: 经典 SC[1]、BISC[7]、确定性方法 [15]、2 比特流和 HSC 的 BN-SB 的 MAE

以二进制计算为基准, 计算 SC 与 BN 之间乘法的偏差. 平均绝对误差 (MAE) 用作精度指标 (MAE 越低表示精度越高). $SC(A, B, N)$ 表示流长度 N 下 $A \times B$ 的 SC 结果, $P(A, B)$ 表示 $A \times B$ 的理论值, 其中 A 为 n 比特, B 为 m 比特. 当 $MAE(N) = 0$ 时, SC 结果与 BN 计算完全一致, 表明实现了零误差计算 [15]

$$MAE(N) = \frac{\sum_{A=0}^{2^n-1} \sum_{B=0}^{2^m-1} |SC(A, B, N) - P(A, B)|}{2^{n+m}}. \tag{15}$$

图7展示了使用不同 RNG 执行 6 比特乘法时四种方法的 MAE. 在比特流方法中, 包括经典方法 [1]、二进制接口 SC (BISC) [7] 和确定性方法的时钟分频 [15], 其中 BISC 具有最佳的渐进精度. BN-SB 实现零误差计算的速度比确定性方法快 2^6 倍. 当 $N < 2^6$ 时, 2 比特流方法的 MAE 优于比特流方法.

逻辑类型		比特流			混合流		二进制
方法		[1]	[7]	[15]	图2	BN-SB	
RNG (n 比特)		LFSR	Sobol	Sobol	LFSR	Sobol	
比特宽度 (n 比特)	4	MAE	0.03	0.03	0.03	0.03	0
		N	16	17	9	6	1
	6	MAE	0.017	0.017	0.016	0.016	0
		N	64	46	39	12	1
	8	MAE	0.008	0.008	0.008	0.008	0
		N	256	122	169	24	1
面积 (n=8)	LUT	18	29	20	20	17	64
	FF	32	40	32	32	24	48
*($\times 10^{-4}$), n=8		2.17	2.8	0.87	2.95	24.5	156.3
功耗 (W) (n=8, 600 MHz)		0.009	0.01	0.009	0.008	0.007	0.018
**(OPS/W)($\times 10^9$)		0.26	0.49	0.12	0.44	3.57	33.3

* 面积效率 = $(\text{LUT} \times N)^{-1}$, N 是流长度, 表示延迟.
** 能耗效率 = $600 \text{ MHz}/(N \cdot \text{功率})$, n=8 比特, OPS 表示每秒乘法操作数.

表 1: 三种方法的延迟 (N)、面积、功耗和效率

表1对比了混合逻辑计算与传统 SC 达到相同精度所需的延迟, 采用 MAE 测量误差. 如 [8] 所述, 由于生成的比特流存在重复, 经典方法 [1] 的精度在流长度 $N > 2^n$ 后不再提高. 因此, 测试中的流长度 N 亦不超过 2^n . 基于经典 SC 在不同比特宽度 BN 上操作所达到的最佳精度, 测试了其他方法达到相同精度所需的流长度.

使用 Kintex-7 FPGA (现场可编程门阵列) 实现了具有 8 比特数据输入的不同方法电路, 所有数据均来自设计工具, 如表1所示. 计算 $(\text{LUT} \times N)^{-1}$ 以测量各方法的面积效率. BN-SB 作为乘法器和 BISC 均存在局限性, 仅适用于多项式操作的第一次乘法, 或需将随机流输出转换为 BN 后方可进行下一次乘法. BN-SB 作为乘法器的面积效率是测试中所有方法中最优的, 达到经典 SC 方法的 11.3 倍. 2 比特混合流与 BISC 的面积效率相近, 相较于经典 SC 提高了约 30%.

4 结论

本文提出了一种称为 HSC 的新数据表示方法. 该方法利用多比特流的期望实现 SC. 与传统 SB 相比, 混合逻辑计算实现了低延迟和低面积占用, 克服了经典方法的长延迟问题. 文中论证了多比特流计算规则的合理性, 并设计了 HSC 的基本电路. 在精度实验中, 使用 BN-SB 作为乘法器将延迟降低至 $1/2^m$ 以达到零误差计算. 使用 8 比特输入数据时, BN-SB 作为乘法器的面积效率达到经典 SC 方法的 11.3 倍.

参考文献

- [1] B. R. Gaines, “Stochastic computing systems,” in *Advances in Information Systems Science*, Boston, MA, USA: Springer, 1969, pp. 37–172.
- [2] A. Alaghi and J. Hayes, “Survey of stochastic computing,” *ACM Trans. Embedded Comput. Syst.*, vol. 12, no. 2s, p. 92, 2013.
- [3] T. Moreau et al., “A taxonomy of general purpose approximate computing techniques,” *IEEE Embedded Syst. Lett.*, vol. 10, no. 1, pp. 2–5, Mar. 2018.
- [4] P. Li, D. J. Lilja, W. Qian, K. Bazargan, and M. D. Riedel, “Computation on stochastic bit streams digital image processing case studies,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 22, no. 3, pp. 449–462, Mar. 2014.
- [5] K. Han, J. Wang, and W. J. Gross, “Bit-wise iterative decoding of polar codes using stochastic computing,” in *Proc. Great Lakes Symp. VLSI (GLSVLSI)*, 2018, pp. 409–414.
- [6] S. Sato, K. Nemoto, S. Akimoto, M. Kinjo, and K. Nakajima, “Implementation of a new neurochip using stochastic logic,” *IEEE Trans. Neural Netw.*, vol. 14, no. 5, pp. 1122–1127, Sep. 2003.
- [7] H. Sim, D. Nguyen, J. Lee, and K. Choi, “Scalable stochastic-computing accelerator for convolutional neural networks,” in *Proc. 22nd Asia South Pacific Des. Auto. Conf. (ASP-DAC)*, Jan. 2017, pp. 696–701.
- [8] M. H. Najafi and D. J. Lilja, “High quality down-sampling for deterministic approaches to stochastic computing,” *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 1, pp. 7–14, Jan.-Mar. 2021.
- [9] K. K. Parhi, “Analysis of stochastic logic circuits in unipolar, bipolar and hybrid formats,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2017, pp. 1–4.
- [10] S. Liu and J. Han, “Toward energy-efficient stochastic circuits using parallel sobol sequences,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 26, no. 7, pp. 1326–1339, Jul. 2018.
- [11] V. Canals, A. Morro, A. Oliver, M. L. Alomar, and J. L. Rosselló, “A new stochastic computing methodology for efficient neural network implementation,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 3, pp. 551–564, Mar. 2016.
- [12] A. Ardakani, F. Leduc-Primeau, N. Onizawa, T. Hanyu, and W. Gross, “VLSI implementation of deep neural network using integral stochastic computing,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 10, pp. 2688–2699, Oct. 2017.



- [13] M. H. Najafi, S. R. Faraji, B. Li, D. J. Lilja, and K. Bazargan, “Accelerating deterministic bit-stream computing with resolution splitting,” in *Proc. 20th Int. Symp. Qual. Electron. Des. (ISQED)*, 2019, pp. 157–162.
- [14] S. R. Faraji and K. Bazargan, “Hybrid binary-unary hardware accelerator,” *IEEE Trans. Comput.*, vol. 69, no. 9, pp. 1308–1319, Sep. 2020.
- [15] M. H. Najafi, D. Jenson, J. Lilja, and D. Riedel, “Performing stochastic computation deterministically,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 12, pp. 2925–2938, Dec. 2019.
- [16] A. Alaghi and J. P. Hayes, “Fast and accurate computation using stochastic circuits,” in *Proc. Des. Automat. Test Eur. Conf. Exhib. (DATE)*, 2014, pp. 1–4.
- [17] Y. Chen and H. Li, “Novel stochastic computing using amplitude and frequency pulse encoding,” in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2022, pp. 656–660.

